2303408050李美琪1030实验

一、瑞幸咖啡小程序修改用户名操作的事件流

用例描述：修改用户名

用例名称	修改用户名
参与者	已登录瑞幸咖啡小程序的用户
前置条件	用户已通过微信授权登录瑞幸咖啡小程序，账号身份验证通过，网络连接正常
后置条件	系统更新用户资料中的用户昵称，订单、积分、优惠券等关联数据同步更新，新昵称即时生效
主事件流	

1. 用户打开瑞幸咖啡小程序，点击底部「我的」进入个人中心；
2. 系统显示个人中心页面，展示当前头像、昵称、会员等级等信息；
3. 用户点击个人头像或昵称区域，进入「个人资料」页面；
4. 系统展示个人资料列表，包含头像、昵称、手机号、生日等可编辑项；
5. 用户点击「昵称」一栏，进入昵称编辑页面；
6. 系统弹出昵称输入框，显示当前昵称，提示昵称长度限制（2-20个字符）；
7. 用户输入新的昵称并点击「完成」按钮；
8. 系统校验昵称格式（长度、敏感词、是否包含特殊字符）；
9. 系统校验通过后更新用户昵称，返回个人资料页面并提示「修改成功」；
10. 用户返回「我的」页面，新昵称即时显示在个人信息栏。

二、图书馆管理系统 UML 图

2.1 用例图（Use Case Diagram）

```
@startuml
left to right direction
actor 读者 as Reader
actor 图书管理员 as Librarian
actor 系统管理员 as Admin
```

```
rectangle 图书馆管理系统 {
  usecase "图书检索" as UC1
  usecase "借书" as UC2
  usecase "还书" as UC3
  usecase "图书预约" as UC4
  usecase "续借" as UC5
  usecase "缴纳罚款" as UC6
  usecase "图书录入" as UC7
  usecase "图书下架" as UC8
  usecase "读者管理" as UC9
  usecase "统计报表" as UC10
  usecase "系统配置" as UC11
}

Reader --> UC1
Reader --> UC2
Reader --> UC3
Reader --> UC4
Reader --> UC5
Reader --> UC6
Librarian --> UC2
Librarian --> UC3
Librarian --> UC7
Librarian --> UC8
Librarian --> UC10
Admin --> UC9
Admin --> UC11
Admin --> UC10
@enduml
```

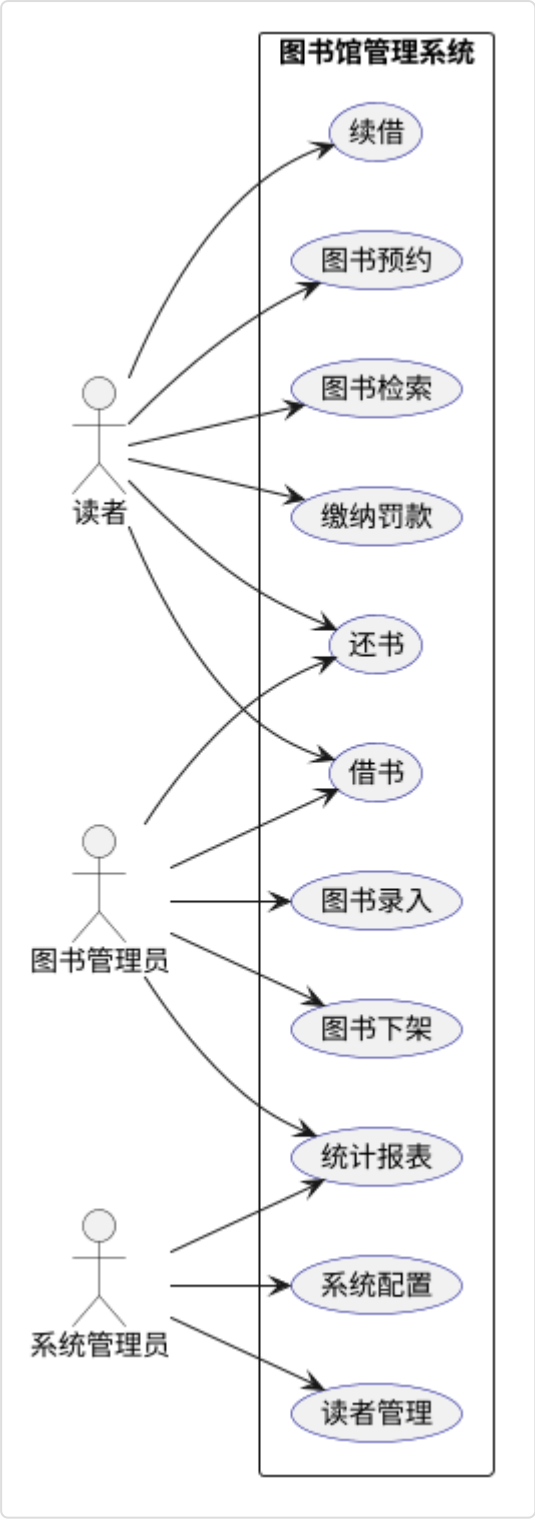


图 2.1 用例图（Use Case Diagram）

2.2 类图（Class Diagram）

```
@startuml
class Reader {
  - readerId: String
  - name: String
  - studentNo: String
```

```
  - phone: String
  - maxBorrowCount: int
  + borrow(): boolean
  + returnBook(): boolean
  + renew(): boolean
}

class Book {
  - bookId: String
  - title: String
  - author: String
  - ISBN: String
  - category: String
  - status: BookStatus
  - location: String
  + updateStatus(): void
}

class BorrowRecord {
  - recordId: String
  - borrowDate: Date
  - dueDate: Date
  - returnDate: Date
  - fineAmount: double
  - renewCount: int
  + calculateFine(): double
}

class Librarian {
  - staffId: String
  - name: String
  - role: String
  + addBook(): boolean
  + removeBook(): boolean
}

class Reservation {
  - reservationId: String
  - reserveDate: Date
  - expiryDate: Date
  - status: ReservationStatus
  + cancel(): boolean
}

class Admin {
  - adminId: String
  - password: String
  + configSystem(): void
}

Reader "1" -- "0..*" BorrowRecord : 产生
Book "1" -- "0..*" BorrowRecord : 被记录
Librarian "1" -- "0..*" Book : 管理
Reader "1" -- "0..*" Reservation : 预约
Book "1" -- "0..*" Reservation : 被预约
Admin -- Librarian : 管理
@enduml
```

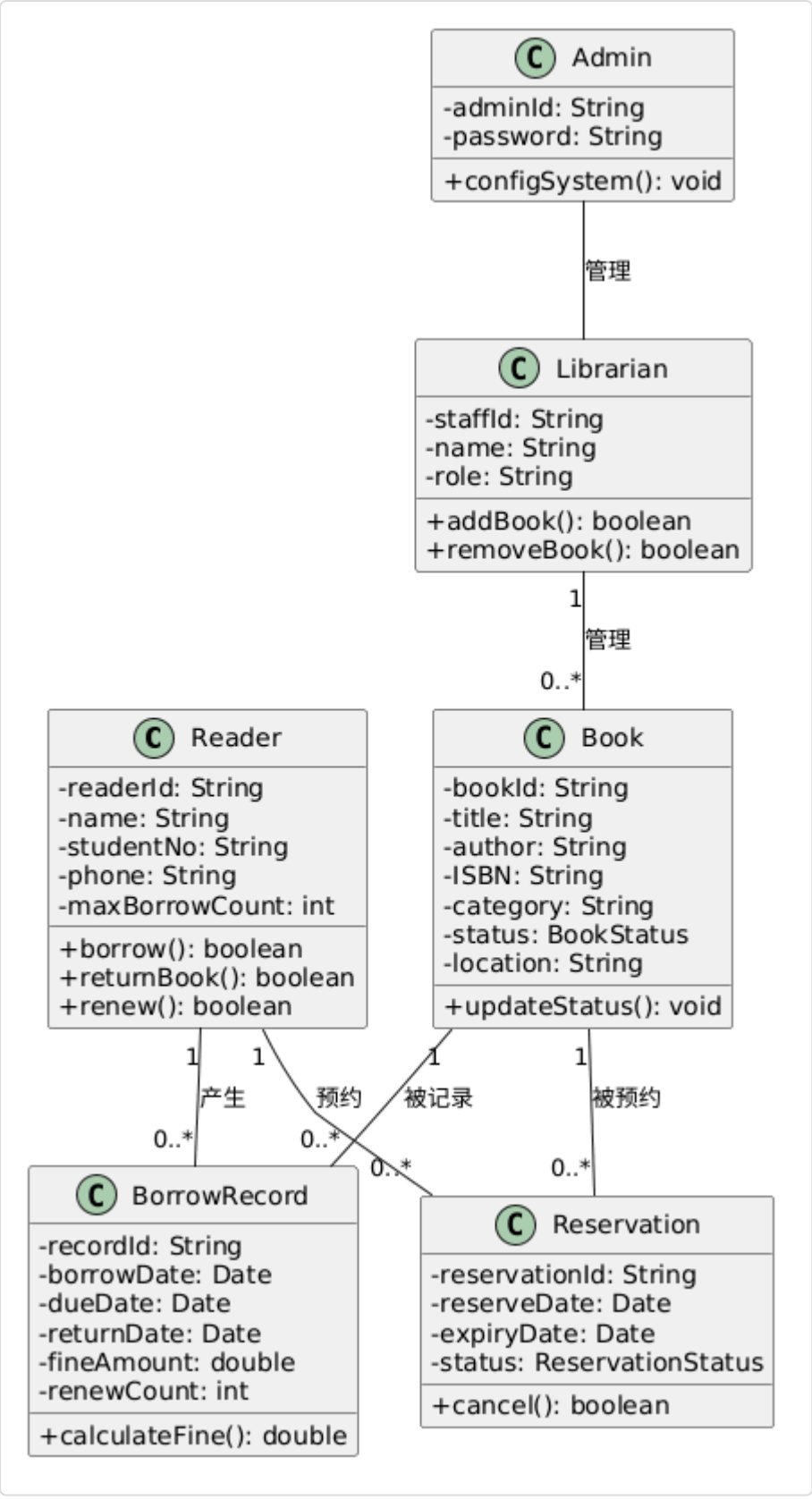


图 2.2 类图（Class Diagram）

2.3 对象图（Object Diagram）

```
@startuml
object "张三" as R001 {
  readerId = "R001"
  name = "张三"
  studentNo = "2026001"
  phone = "138****1234"
  maxBorrowCount = 5
}

object "三体" as B001 {
  bookId = "B001"
  title = "三体"
  author = "刘慈欣"
  ISBN = "9787536692930"
  category = "科幻小说"
  status = "在馆"
}

object "活着" as B002 {
  bookId = "B002"
  title = "活着"
  author = "余华"
  ISBN = "9787506365437"
  category = "当代文学"
  status = "借出"
}

object "BR001" as BR001 {
  recordId = "BR001"
  borrowDate = "2026-10-20"
  dueDate = "2026-11-03"
  returnDate = null
  fineAmount = 0.0
  renewCount = 0
}

object "李管理员" as L001 {
  staffId = "L001"
  name = "李管理员"
  role = "前台"
}

R001 -- BR001 : 借书记录
B002 -- BR001 : 被借出
L001 -- B001 : 录入
@enduml
```

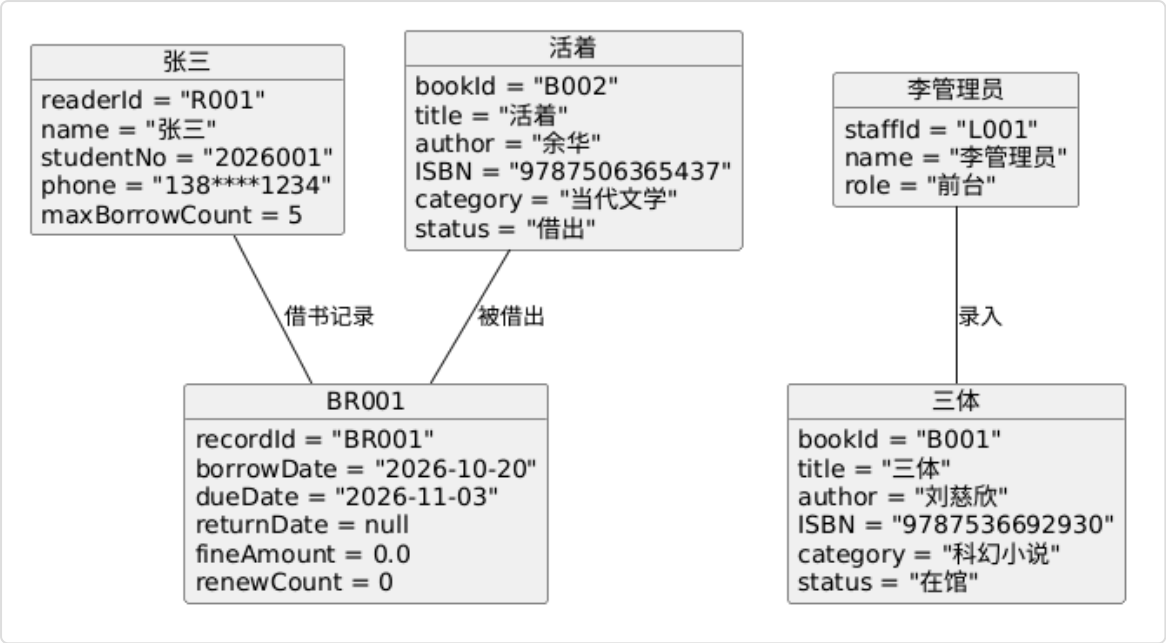


图 2.3 对象图（Object Diagram）

2.4 时序图（Sequence Diagram）——借书流程

```
@startuml
autonumber
actor 读者
actor 图书管理员
participant "借书系统" as System
database "数据库" as DB

读者 -> 图书管理员: 出示读者证，提出借书
图书管理员 -> System: 扫描读者证
System -> DB: 查询读者信息
DB --> System: 返回读者资料与借阅状态
System -> System: 校验读者是否可借
图书管理员 -> System: 扫描图书条形码
System -> DB: 查询图书信息
DB --> System: 返回图书状态

alt 图书可借
  System -> DB: 创建借书记录
  System -> DB: 更新图书状态为借出
  System --> 图书管理员: 显示借书成功，应还日期
  图书管理员 --> 读者: 交付图书，告知还书日期
else 图书不可借
  System --> 图书管理员: 提示图书不可借原因
  图书管理员 --> 读者: 告知无法借书
end
@enduml
```

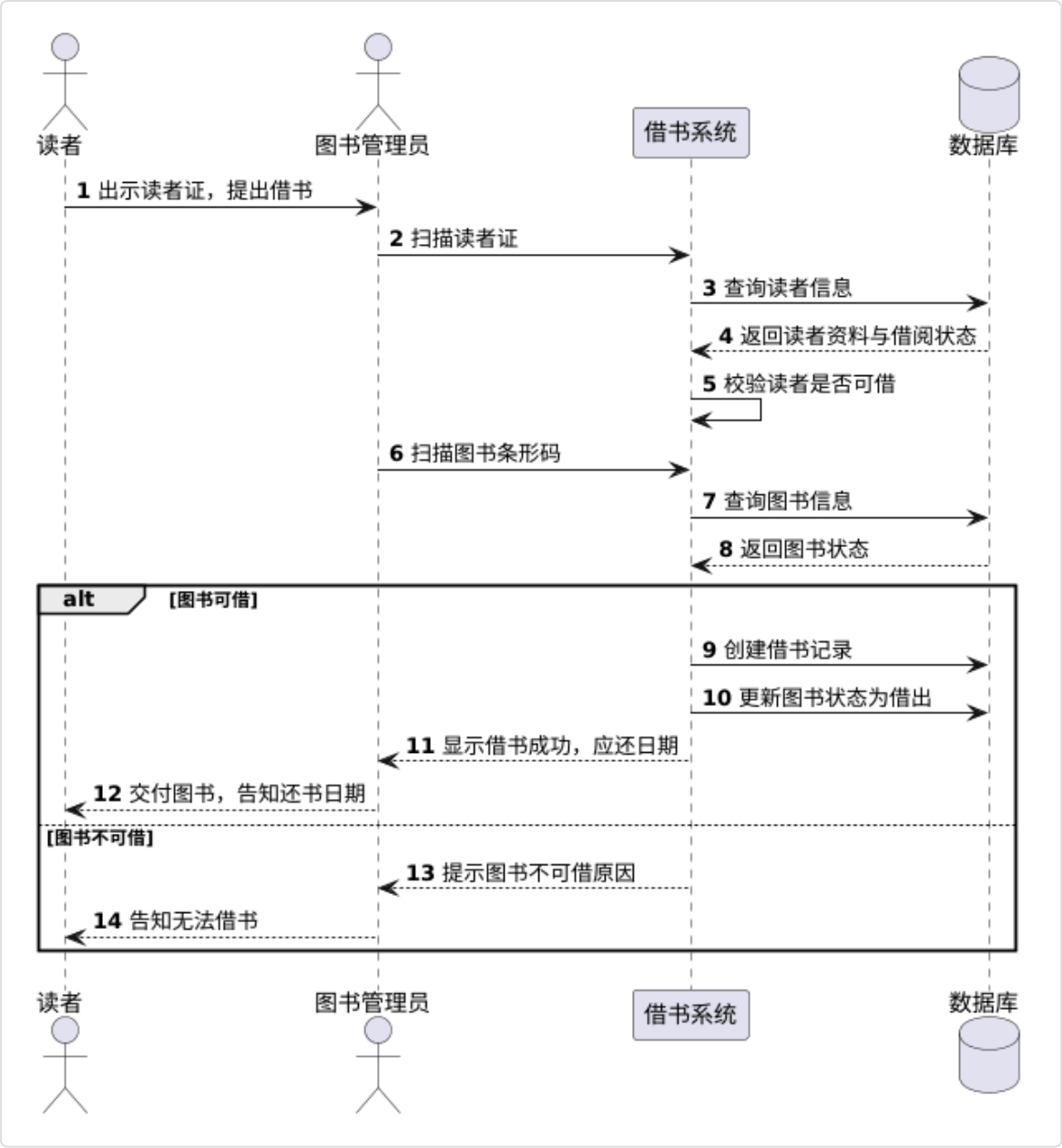


图 2.4 时序图（Sequence Diagram）——借书流程

2.5 活动图（Activity Diagram）——还书流程

```
@startuml
start
:读者提交归还图书;
:图书管理员扫描图书条形码;
:系统查询图书借阅记录;
if (是否逾期?) then (是)
  :系统计算逾期罚款金额;
  :管理员收取罚款;
else (否)
endif
:系统更新借书记录（归还日期）;
:更新图书状态为在馆;
if (该书是否有预约?) then (是)
```

```
  :系统通知预约读者;
  :图书状态标记为预约保留;
else (否)
  :图书状态标记为可借;
endif
:打印还书凭证;
:读者离开;
stop
@enduml
```

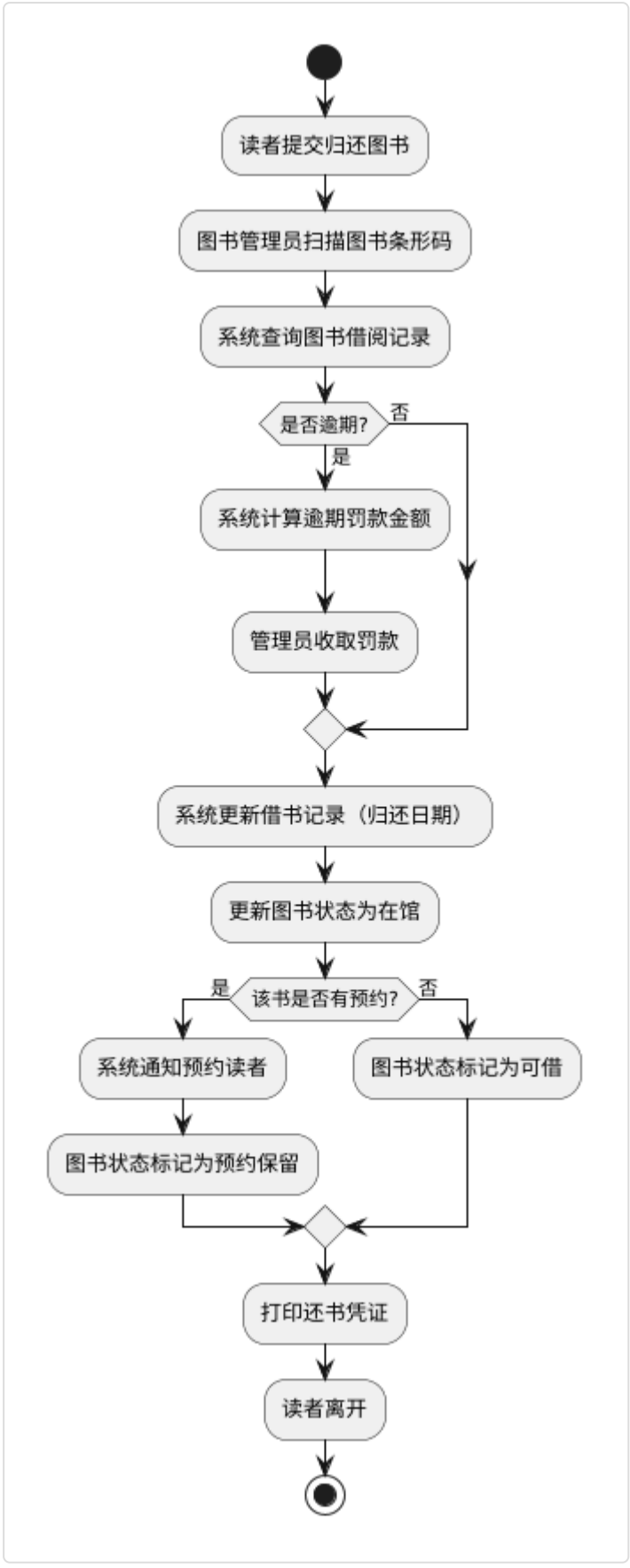


图 2.5 活动图（Activity Diagram）——还书流程

2.6 状态图（State Diagram）——图书状态

```
@startuml
[*] --> 在馆 : 图书录入
在馆 --> 借出 : 读者借书
在馆 --> 预约保留 : 读者预约
预约保留 --> 借出 : 预约读者到馆借书
预约保留 --> 在馆 : 预约过期或取消
借出 --> 在馆 : 读者归还
借出 --> 预约保留 : 归还后有预约
在馆 --> 下架 : 管理员下架
下架 --> [*] : 注销处理
@enduml
```

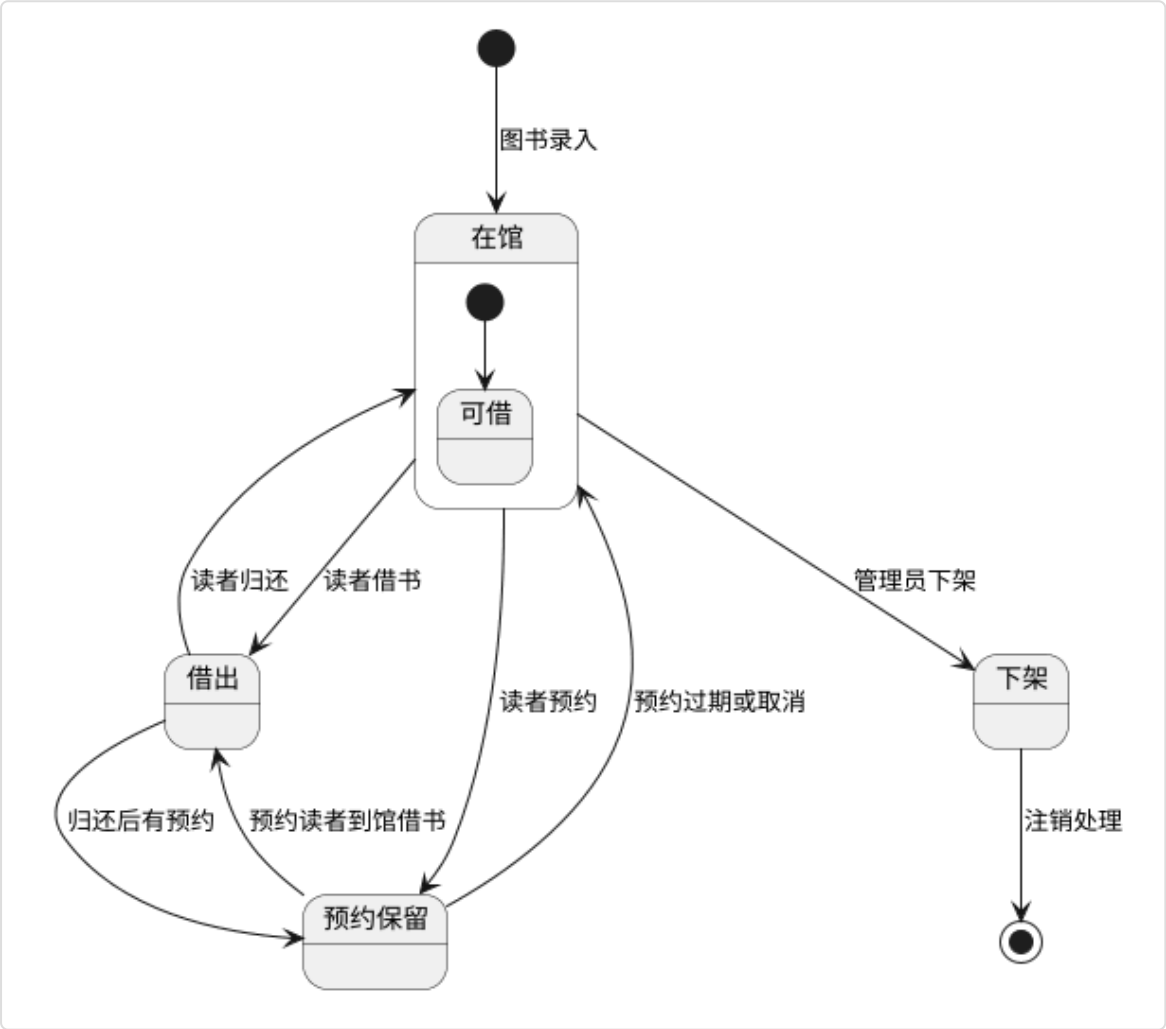


图 2.6 状态图（State Diagram）——图书状态

2.7 组件图（Component Diagram）

```
@startuml
package "用户界面层" {
  [读者端小程序] as MiniApp
  [管理员端Web] as AdminWeb
}
```

```
package "业务逻辑层" {
  [读者管理组件] as ReaderComp
  [图书管理组件] as BookComp
  [借阅管理组件] as BorrowComp
  [罚款计算组件] as FineComp
}

package "数据访问层" {
  [ReaderDAO] as ReaderDAO
  [BookDAO] as BookDAO
  [BorrowDAO] as BorrowDAO
}

package "外部接口" {
  [短信通知服务] as SMS
  [微信支付接口] as WeChatPay
}

database "数据库" as DB

MiniApp --> ReaderComp
AdminWeb --> ReaderComp
AdminWeb --> BookComp
MiniApp --> BorrowComp
ReaderComp --> ReaderDAO
BookComp --> BookDAO
BorrowComp --> BorrowDAO
FineComp --> WeChatPay
BorrowComp --> SMS
ReaderDAO --> DB
BookDAO --> DB
BorrowDAO --> DB
@enduml
```

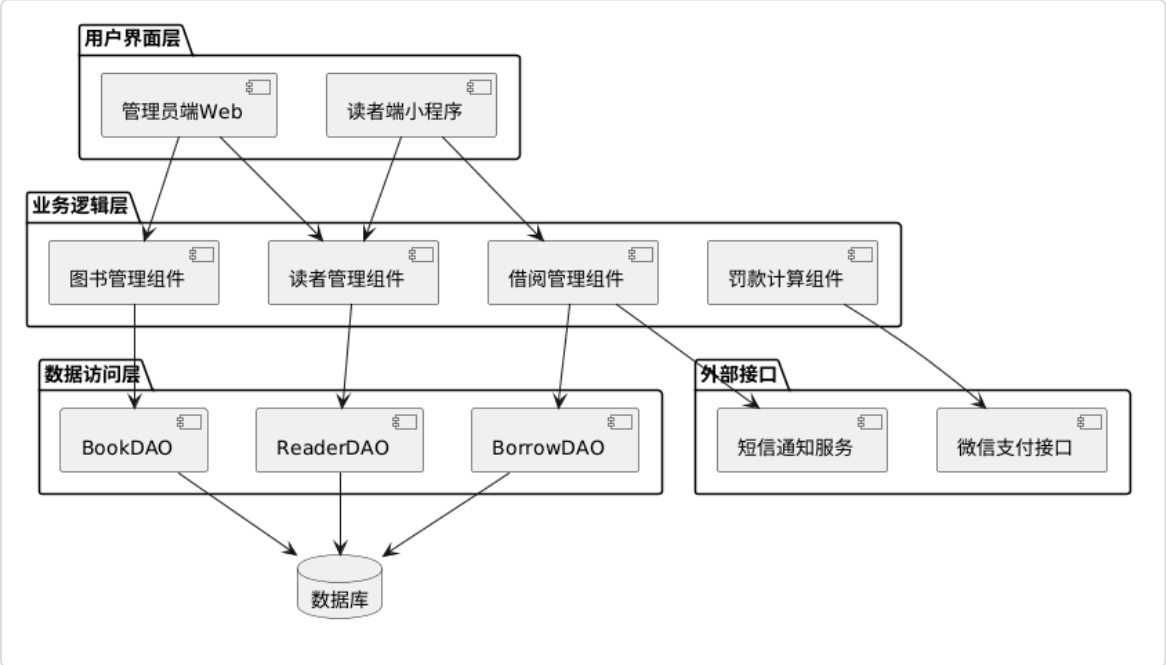


图 2.7 组件图（Component Diagram）

2.8 部署图（Deployment Diagram）

```
@startuml
node "读者手机端" as Phone {
  artifact "微信小程序" as MiniAppArt
}

node "管理员终端" as AdminPC {
  artifact "Web 浏览器" as BrowserArt
}

node "云服务器\n(Web/应用服务器)" as Server {
  artifact "Nginx 反向代理" as NginxArt
  artifact "Spring Boot 应用" as SpringArt
}

node "数据库服务器" as DBServer {
  artifact "MySQL 8.0" as MySQLArt
}

node "第三方服务" as ThirdParty {
  artifact "微信开放平台" as WeChatArt
  artifact "短信服务商" as SMSArt
}

Phone --> Server : HTTPS
AdminPC --> Server : HTTP
Server --> DBServer : JDBC
Server --> ThirdParty : HTTPS API
@enduml
```

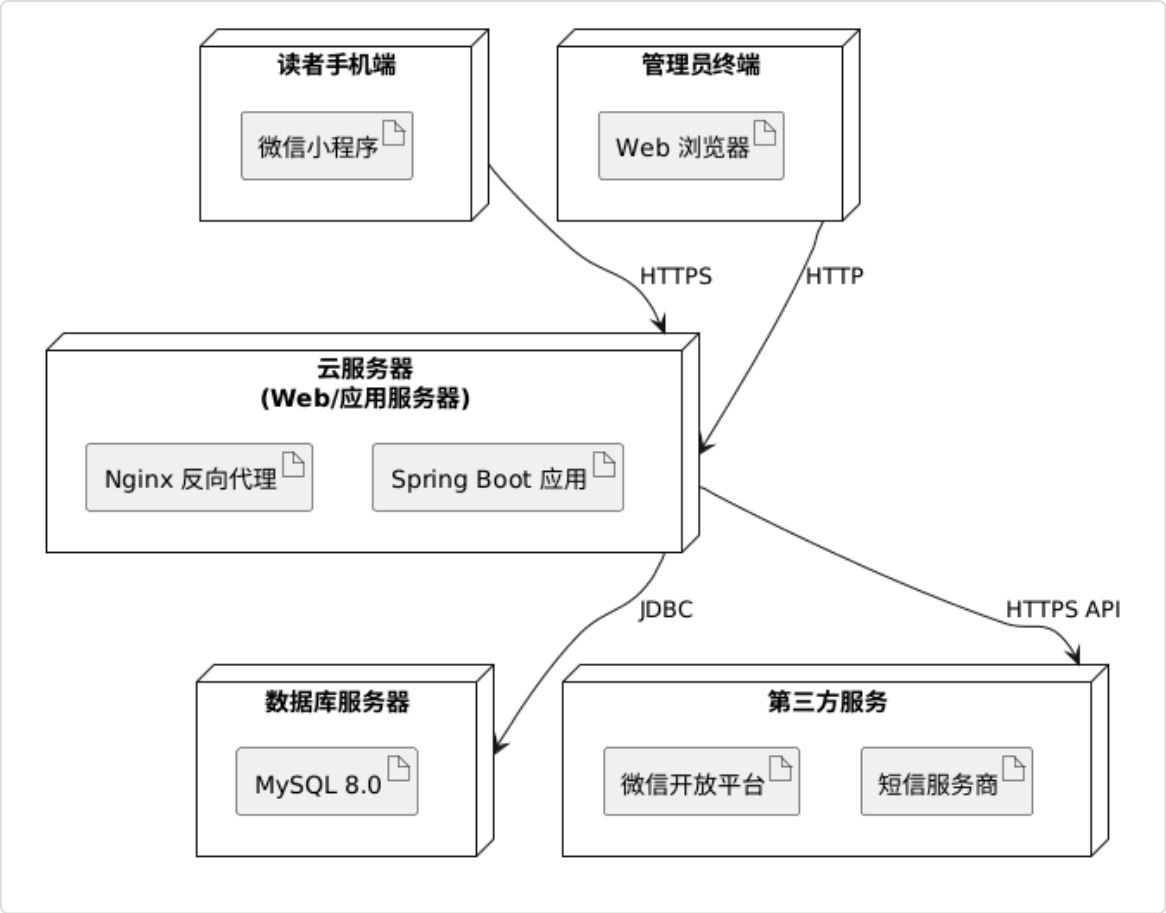


图 2.8 部署图（Deployment Diagram）

2.9 定时图（Timing Diagram）——借书操作时间约束

```
@startuml
robust "读者端" as Reader
robust "管理员端" as Librarian
robust "系统响应" as System

@0
Reader is 空闲
Librarian is 空闲
System is 空闲

@2
Reader is 出示读者证
Librarian is 扫描读者证
System is 读者信息校验

@3
Reader is 出示读者证
Librarian is 扫描图书
System is 图书信息查询

@4
Reader is 出示读者证
Librarian is 确认借书
```

```
System is 创建借书记录

@5
Reader is 接收图书
Librarian is 确认借书
System is 结果返回

@7
Reader is 接收图书
Librarian is 空闲
System is 空闲
@enduml
```

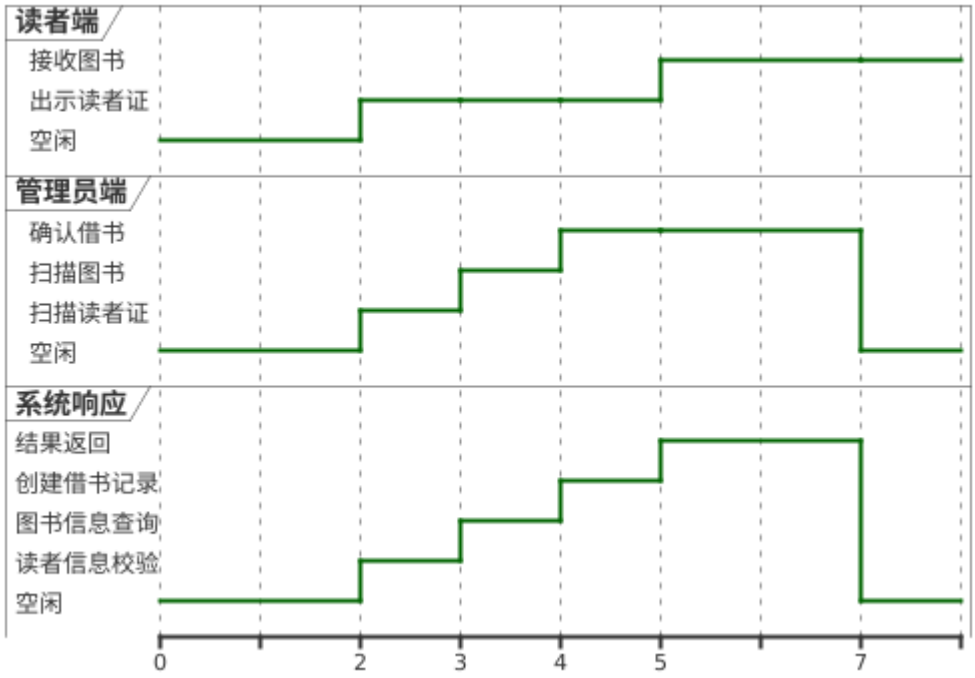


图 2.9 定时图（Timing Diagram）——借书操作时间约束